

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

MPI Programming with Slurm

This laboratory introduces MPI programming using the *Slurm* workload manager. We will use the departmental *Pine* cluster, which features 16 Dell PowerEdge R730 nodes and a high-speed, low-latency network. Each server includes the following hardware:

- Dual Intel Xeon E5-2680 v3 CPUs providing 24 physical cores and 60MB of L3 cache for optimized parallel data handling.
- 256GB of DDR4 RDIMM RAM and 2TB HDD.
- Low latency 40Gbps Mellanox InfiniBand backbone paired with gigabit ethernet ports for cluster management.

By the end of this session, you will understand how to write distributed-memory programs and use Slurm to orchestrate their execution across this hardware.

Environment Setup

Before writing code, let us set up a workspace on the cluster:

1. Log in to the head node using your Trinity user ID:

```
$ ssh userid@pine.cs.trincoll.edu
```

When asked for a password, enter 'cpsc375'.

2. Change your password:

```
[pine] $ passwd
```

3. Create a workspace:

```
[pine] $ mkdir mpilab && cd mpilab
```

Writing the MPI Code

Write a basic C program that identifies which rank (process) is running on which hostname (physical node).

1. Create the file: `mpi_hello.c`.
2. Paste the following code:

```
#include <mpi.h>
#include <stdio.h>
```

```
#include <unistd.h>

int main(int argc, char* argv[]) {
    MPI_Init(NULL, NULL);

    int size, rank;
    char hostname[256];

    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    gethostname(hostname, 256);

    printf("Hello from Rank %d out of %d running on %s\n", rank, size, hostname);

    MPI_Finalize();
    return 0;
}
```

3. Compile:

```
$ mpicc mpi_hello.c -o mpi_hello
```

Job Submission Using Slurm

Instead of running the code directly, you must request resources from the Slurm scheduler.

1. Create a batch script: submit.sh

```
#!/bin/bash
#SBATCH --job-name=hello      # Name in the queue
#SBATCH --nodes=3            # Request 3 nodes
#SBATCH --ntasks-per-node=1  # 1 process per node
#SBATCH --output=result_%j.log # Save output to this file (%j = JobID)

# Run the MPI program you compiled
srun ./mpi_hello
```

2. Submit to the queue:

```
$ sbatch submit.sh
```

3. Check status:

```
$ squeue
```

4. View results: Once the job finishes, check result_*[jobid]*.txt.

5. Repeat with different values for the number nodes ($n \leq 16$) and the number of tasks per node (each Pine node has 24 physical cores).

NOTE: You may find the following Slurm commands useful:

- `scancel jobID` – signals or cancels jobs, e.g., when you just realized your code has a bug and wants to kill it (find the Job ID using `squeue`).
- `sacct` – displays accounting data for all jobs, e.g., if you wonder why your job failed last night.

Finally, what if you ask for more resources than the physical hardware provides?

1. Try to submit a job requesting more cores than a single node has (e.g., `--ntasks-per-node=48`).
2. Monitor the status using `squeue`. What does it say about your job?
3. Cancel your job using `scancel`.

- **Welcome: Sean**

- [LogOut](#)

